AegisOps — Full Project Development Guide

Autonomous AI DevOps War-Room for Zero-Downtime Systems

Table of Contents

---

1. Project Overview

AegisOps is an AI-powered autonomous Site Reliability Engineering (SRE) platform. It replaces the traditional reactive DevOps workflow with a proactive self-healing system built on specialized AI agents.

Core Capabilities

Capability	**Description**
Monitoring	Real-time ingestion of logs, metrics, and traces
Anomaly Detection	ML-based detection of unusual system patterns
Failure Prediction	Forecast outages before they happen

Capability	Description
Root Cause Analysis	Auto-correlate logs, metrics, and deployment history
AI Recovery	Generate and execute recovery plans
Validation	Test fixes before deploying to production
Learning	Build a knowledge base from past incidents

---

2. Folder Structure

aegisops/

```
├── backend/
│   ├── main.py                # FastAPI entry point
│   ├── routers/
│   │   ├── incidents.py       # Incident CRUD APIs
│   │   ├── metrics.py         # Metrics ingestion APIs
│   │   ├── agents.py          # Agent trigger endpoints
│   │   └── predictions.py     # Prediction result APIs
│   ├── agents/
│   │   ├── monitoring_agent.py
│   │   ├── anomaly_agent.py
│   │   ├── prediction_agent.py
│   │   ├── rca_agent.py
│   │   ├── recovery_agent.py
```

```
│   │   ├── testing_agent.py
│   │   └── security_agent.py
│   ├── models/
│   │   ├── incident.py          # SQLAlchemy models
│   │   ├── metric.py
│   │   └── recovery_action.py
│   ├── services/
│   │   ├── prometheus_client.py # Fetch metrics from Prometheus
│   │   ├── rag_service.py       # RAG knowledge base
│   │   └── llm_service.py       # OpenAI/Gemini wrapper
│   ├── db.py                    # PostgreSQL connection
│   └── config.py                # Settings / env vars
│
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Dashboard.tsx        # Executive dashboard
│   │   │   ├── IncidentCommand.tsx   # Live incident view
│   │   │   ├── Predictions.tsx       # Failure forecast
│   │   │   └── AIRecommendations.tsx # AI action panel
│   │   ├── components/
│   │   │   ├── SystemHealthCard.tsx
```

```
│   │   │   ├── IncidentTimeline.tsx
│   │   │   ├── AgentStatusPanel.tsx
│   │   │   └── RecoveryActionCard.tsx
│   │   ├── hooks/
│   │   │   └── useWebSocket.ts       # Live updates
│   │   └── App.tsx
│
├── ml/
│   ├── anomaly/
│   │   ├── isolation_forest.py
│   │   ├── autoencoder.py
│   │   └── lstm_detector.py
│   └── prediction/
│       └── failure_predictor.py
│
├── infra/
│   ├── docker-compose.yml
│   ├── prometheus.yml
│   └── k8s/
│       ├── backend-deployment.yaml
│       └── frontend-deployment.yaml
```

```
│
├── scripts/
│   ├── simulate_failure.py      # Inject fake failures for demo
│   └── seed_incidents.py        # Populate DB with sample data
│
└── README.md
```

---

3. Tech Stack Breakdown

Frontend

Tool	**Purpose**
React + TypeScript	UI framework
Tailwind CSS	Styling
Recharts	Graphs and dashboards
Axios	API calls
WebSocket / Socket.IO	Live real-time updates

Backend

Tool	**Purpose**
FastAPI	REST API server
Python 3.11+	Core language
SQLAlchemy	ORM for PostgreSQL
Pydantic	Request/response validation
Celery	Background agent task queue

Database

Tool	Purpose
PostgreSQL	Main relational database
Redis	Cache + Celery broker
FAISS / Pinecone	Vector store for RAG

AI / ML

Tool	Purpose
LangGraph	Multi-agent orchestration
CrewAI	Agent collaboration framework
OpenAI GPT-4 / Gemini	LLM for RCA and recovery generation
Sentence Transformers	Embedding for RAG
scikit-learn	Isolation Forest
TensorFlow / PyTorch	LSTM, Autoencoder models
LangChain	RAG pipeline

Monitoring

Tool	Purpose
Prometheus	Metrics scraping
Grafana	Visualization
OpenTelemetry	Distributed tracing

DevOps / Cloud

Tool	Purpose
Docker	Containerization
Docker Compose	Local multi-service setup

Tool	Purpose
Kubernetes	Production orchestration
GitHub Actions	CI/CD pipeline
AWS / GCP / Azure	Cloud hosting

---

4. Phase-wise Development Plan

Phase 1 — Foundation (Week 1-2)

- ☐ Set up project repository and folder structure
- ☐ Configure Docker Compose with PostgreSQL, Redis, FastAPI
- ☐ Create database models (Incidents, Metrics, Recovery Actions)
- ☐ Build basic FastAPI server with health check endpoint
- ☐ Set up React frontend with Tailwind CSS and routing
- ☐ Deploy Prometheus and Grafana locally
- ☐ Connect FastAPI to PostgreSQL using SQLAlchemy

Phase 2 — Monitoring & Anomaly Detection (Week 3-4)

- ☐ Build Monitoring Agent — poll Prometheus every 30 seconds
- ☐ Create metrics ingestion API endpoint
- ☐ Implement Isolation Forest model for anomaly detection
- ☐ Train LSTM on synthetic time-series data
- ☐ Store detected anomalies in PostgreSQL
- ☐ Build System Health Card component in React
- ☐ Create WebSocket connection for real-time metric updates

Phase 3 — AI Agents — RCA & Recovery (Week 5-6)

- ☐ Integrate OpenAI / Gemini API
- ☐ Build Root Cause Analysis Agent using LangGraph
- ☐ Build log correlation pipeline (logs + metrics + deployments)
- ☐ Implement RAG service using FAISS + Sentence Transformers
- ☐ Build Recovery Agent with action generation
- ☐ Create Incident Command Center page in frontend
- ☐ Implement AI Recommendations Panel

Phase 4 — Prediction & Testing Agent (Week 7)

- ☐ Build Failure Prediction Agent using historical data
- ☐ Create Failure Prediction Dashboard page
- ☐ Build Testing Agent to run validation before deployment
- ☐ Add Security Agent for anomaly monitoring
- ☐ Implement autonomous vs recommendation mode toggle

Phase 5 — Demo Polish & Deployment (Week 8)

- ☐ Build failure simulation script
- ☐ Create full demo walkthrough
- ☐ Deploy to cloud (AWS / GCP)
- ☐ Dockerize all services
- ☐ Write documentation and README
- ☐ Record demo video

---

5. Backend Development

FastAPI Setup (main.py)

```
from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from routers import incidents, metrics, agents, predictions

app = FastAPI(title="AegisOps API", version="1.0.0")

app.add_middleware(

    CORSMiddleware,

    allow_origins=["*"],

    allow_methods=["*"],

    allow_headers=["*"],
```

```
)

app.include_router(incidents.router, prefix="/api/incidents")

app.include_router(metrics.router, prefix="/api/metrics")

app.include_router(agents.router, prefix="/api/agents")

app.include_router(predictions.router, prefix="/api/predictions")

@app.get("/health")

def health_check():

    return {"status": "healthy", "service": "AegisOps"}
```

Database Models (models/incident.py)

```
from sqlalchemy import Column, Integer, String, Float, DateTime, Text, Enum

from sqlalchemy.sql import func

from db import Base

import enum

class IncidentStatus(str, enum.Enum):

    OPEN = "open"

    INVESTIGATING = "investigating"

    RESOLVED = "resolved"

class Incident(Base):

    __tablename__ = "incidents"

    id = Column(Integer, primary_key=True, index=True)

    title = Column(String(255), nullable=False)

    service = Column(String(100), nullable=False)
```

```
    status = Column(Enum(IncidentStatus), default=IncidentStatus.OPEN)
    severity = Column(String(20))       # critical / high / medium / low
    root_cause = Column(Text)
    confidence_score = Column(Float)
    recovery_action = Column(Text)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    resolved_at = Column(DateTime(timezone=True), nullable=True)
```

Prometheus Client (services/prometheus_client.py)

```
import httpx
from config import settings

PROMETHEUS_URL = settings.PROMETHEUS_URL

async def fetch_metric(query: str) -> dict:
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"{PROMETHEUS_URL}/api/v1/query",
            params={"query": query}
        )
        return response.json()

async def get_cpu_usage(service: str) -> float:
    result = await fetch_metric(
        f'rate(process_cpu_seconds_total{{job="{service}"}}[5m])'
```

```
    )
    return float(result["data"]["result"][0]["value"][1])
async def get_error_rate(service: str) -> float:
    result = await fetch_metric(
        f'rate(http_requests_total{{job="{service}",status=~"5.."}}[5m])'
    )
    return float(result["data"]["result"][0]["value"][1])
```

---

6. AI Agent Development

How Agents Work Together (LangGraph)

```
Monitoring Agent
      |
      ▼
Anomaly Detection Agent --> No anomaly --> Continue monitoring
      |
   Anomaly Found
      |
      ▼
Root Cause Analysis Agent
      |
      ▼
```

Recovery Agent

|

├── Recommendation Mode --> Human approves --> Execute

└── Autonomous Mode --> Auto execute

|

▼

Testing Agent (validate fix)

|

▼

Deployment Controller

Root Cause Analysis Agent (agents/rca_agent.py)

```
from langchain_openai import ChatOpenAI

from langchain.prompts import PromptTemplate

from services.rag_service import search_similar_incidents

llm = ChatOpenAI(model="gpt-4", temperature=0)

RCA_PROMPT = PromptTemplate(

    input_variables=["logs", "metrics", "recent_deployments", "similar_incidents"],

    template="""

    You are a senior Site Reliability Engineer performing root cause analysis.

    Recent Logs:

    {logs}

    System Metrics:
```

```
    {metrics}

    Recent Deployments:

    {recent_deployments}

    Similar Past Incidents:

    {similar_incidents}

    Analyze the above and provide:

    1. Root Cause (specific and technical)

    2. Confidence Score (0-100)

    3. Impact Analysis

    4. Recommended Recovery Actions (ranked by priority)

    Respond in JSON format.

    """

)

async def run_rca(logs: str, metrics: dict, deployments: list) -> dict:

    similar = await search_similar_incidents(logs)

    chain = RCA_PROMPT | llm

    result = await chain.ainvoke({

        "logs": logs,

        "metrics": str(metrics),

        "recent_deployments": str(deployments),

        "similar_incidents": str(similar)
```

```
    })
    return result
```

Anomaly Detection (ml/anomaly/isolation_forest.py)

```
from sklearn.ensemble import IsolationForest
import numpy as np
import joblib
class AnomalyDetector:
    def __init__(self):
        self.model = IsolationForest(
            n_estimators=100,
            contamination=0.05,   # 5% expected anomalies
            random_state=42
        )
    def train(self, data: np.ndarray):
        self.model.fit(data)
        joblib.dump(self.model, "anomaly_model.pkl")
    def predict(self, data_point: np.ndarray) -> bool:
        prediction = self.model.predict([data_point])
        return prediction[0] == -1  # -1 means anomaly
    def load(self):
        self.model = joblib.load("anomaly_model.pkl")
# Usage
```

```
detector = AnomalyDetector()

# Features: [cpu_usage, memory_usage, error_rate, latency_ms, request_count]

sample = np.array([0.85, 0.90, 0.15, 2500, 1200])

is_anomaly = detector.predict(sample)
```

RAG Service (services/rag_service.py)

```
from sentence_transformers import SentenceTransformer

import faiss

import numpy as np

import json

model = SentenceTransformer("all-MiniLM-L6-v2")

index = faiss.IndexFlatL2(384)   # 384-dim embeddings

incident_store = []

def add_incident_to_knowledge_base(incident: dict):

    text = f"{incident['title']} {incident['root_cause']} {incident['recovery_action']}"

    embedding = model.encode([text])[0]

    index.add(np.array([embedding]))

    incident_store.append(incident)

async def search_similar_incidents(query: str, top_k: int = 3) -> list:

    query_embedding = model.encode([query])[0]

    distances, indices = index.search(np.array([query_embedding]), top_k)

    return [incident_store[i] for i in indices[0] if i < len(incident_store)]
```

---

7. Frontend Development

Dashboard Page (pages/Dashboard.tsx)

```
import { useState, useEffect } from "react";

import SystemHealthCard from "../components/SystemHealthCard";

import IncidentTimeline from "../components/IncidentTimeline";

import AgentStatusPanel from "../components/AgentStatusPanel";

import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";

export default function Dashboard() {

  const [healthScore, setHealthScore] = useState(98);

  const [incidents, setIncidents] = useState([]);

  const [metrics, setMetrics] = useState([]);

  useEffect(() => {

    // Fetch data from API

    fetch("/api/incidents?status=open")

      .then(r => r.json())

      .then(setIncidents);

  }, []);

  return (

    <div className="p-6 bg-gray-950 min-h-screen text-white">

      <h1 className="text-3xl font-bold mb-6">AegisOps War Room</h1>
```

```
<div className="grid grid-cols-4 gap-4 mb-8">
  <SystemHealthCard score={healthScore} />
  <StatCard label="Active Incidents" value={incidents.length} color="red" />
  <StatCard label="Resolved Today" value={12} color="green" />
  <StatCard label="Uptime" value="99.97%" color="blue" />
</div>
<div className="grid grid-cols-2 gap-6">
  <div className="bg-gray-900 rounded-xl p-4">
    <h2 className="text-xl font-semibold mb-4">System Metrics</h2>
    <ResponsiveContainer width="100%" height={250}>
      <LineChart data={metrics}>
        <XAxis dataKey="time" />
        <YAxis />
        <Tooltip />
        <Line type="monotone" dataKey="cpu" stroke="#ef4444" />
        <Line type="monotone" dataKey="memory" stroke="#3b82f6" />
        <Line type="monotone" dataKey="errorRate" stroke="#f59e0b" />
      </LineChart>
    </ResponsiveContainer>
  </div>
  <IncidentTimeline incidents={incidents} />
</div>
```

```
    </div>
  );
}
```

WebSocket Hook (hooks/useWebSocket.ts)

```
import { useEffect, useState } from "react";

export function useWebSocket(url: string) {
  const [data, setData] = useState(null);
  const [connected, setConnected] = useState(false);
  useEffect(() => {
    const ws = new WebSocket(url);
    ws.onopen = () => setConnected(true);
    ws.onmessage = (event) => setData(JSON.parse(event.data));
    ws.onclose = () => setConnected(false);
    return () => ws.close();
  }, [url]);
  return { data, connected };
}

// Usage in component:
// const { data } = useWebSocket("ws://localhost:8000/ws/metrics");
```

---

8. Database Design

Tables

incidents

Column	Type	Description
id	INTEGER PK	Auto increment
title	VARCHAR(255)	Incident name
service	VARCHAR(100)	Affected service
status	ENUM	open / investigating / resolved
severity	VARCHAR(20)	critical / high / medium / low
root_cause	TEXT	AI-identified root cause
confidence_score	FLOAT	0.0 to 1.0
recovery_action	TEXT	AI-generated fix
created_at	TIMESTAMP	Auto set
resolved_at	TIMESTAMP	When resolved

metrics

Column	Type	Description
id	INTEGER PK	Auto increment
service	VARCHAR(100)	Service name
cpu_usage	FLOAT	0.0 to 1.0
memory_usage	FLOAT	0.0 to 1.0
error_rate	FLOAT	Errors per second
latency_ms	FLOAT	Response time
request_count	INTEGER	Requests per minute

Column	Type	Description
is_anomaly	BOOLEAN	Flagged by ML model
recorded_at	TIMESTAMP	Time of reading

recovery_actions

Column	Type	Description
id	INTEGER PK	Auto increment
incident_id	INTEGER FK	Linked incident
action_type	VARCHAR(50)	restart / rollback / scale / config
description	TEXT	What the action does
status	ENUM	pending / approved / executed / failed
executed_at	TIMESTAMP	When executed
executed_by	VARCHAR(50)	autonomous / human name

---

9. Monitoring & Infrastructure

Docker Compose (docker-compose.yml)

```
version: "3.9"

services:

  backend:

    build: ./backend

    ports:

      - "8000:8000"
```

```
  environment:
    - DATABASE_URL=postgresql://postgres:password@db:5432/aegisops
    - REDIS_URL=redis://redis:6379
  depends_on:
    - db
    - redis
frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend
db:
  image: postgres:15
  environment:
    POSTGRES_DB: aegisops
    POSTGRES_PASSWORD: password
  volumes:
    - postgres_data:/var/lib/postgresql/data
redis:
  image: redis:7
  ports:
```

```
      - "6379:6379"

  prometheus:

    image: prom/prometheus

    volumes:

      - ./infra/prometheus.yml:/etc/prometheus/prometheus.yml

    ports:

      - "9090:9090"

  grafana:

    image: grafana/grafana

    ports:

      - "3001:3000"

    depends_on:

      - prometheus

volumes:

  postgres_data:
```

Prometheus Config (infra/prometheus.yml)

```
global:

  scrape_interval: 15s

scrape_configs:

  - job_name: "aegisops-backend"

    static_configs:
```

```
    - targets: ["backend:8000"]

  - job_name: "sample-service"

    static_configs:

      - targets: ["sample-service:8080"]
```

---

10. Demo Scenario Setup

This is the most important part for judges. Follow these steps exactly.

Step 1 — Start All Services

```
docker-compose up -d
```

Step 2 — Seed Sample Data

```
python scripts/seed_incidents.py
```

Step 3 — Show Healthy Dashboard

Open http://localhost:3000 and show the green System Health score (98/100).

Step 4 — Inject a Failure

```
python scripts/simulate_failure.py --type database_connection_leak
```

This script will:

- Spike memory usage metrics
- Inject fake error logs (connection pool exhausted)
- Push to Prometheus metrics endpoint

Step 5 — Watch AegisOps React

- Dashboard turns red within 30 seconds
- Anomaly Detection Agent flags the metrics spike
- RCA Agent runs and outputs: **"Root Cause: Connection Pool Exhaustion — Confidence: 95%"**
- Recovery Agent suggests: **"Restart service + Increase pool size"**

Step 6 — Execute Recovery

Click "Approve & Execute" in the AI Recommendations Panel.

Step 7 — System Heals

- Service restarts automatically
- Dashboard returns to green
- Incident report generated and saved

Step 8 — Show Incident Report

Navigate to the Incident History page and show the auto-generated incident report with full timeline.

Failure Simulation Script (scripts/simulate_failure.py)

```
import argparse

import httpx

import time

import random

def simulate_database_connection_leak():

    print("Simulating database connection leak...")

    for i in range(20):

        # Push fake high-memory, high-error-rate metrics

        payload = {

            "service": "payment-service",

            "cpu_usage": random.uniform(0.7, 0.95),

            "memory_usage": random.uniform(0.85, 0.99),

            "error_rate": random.uniform(0.3, 0.8),
```

```
            "latency_ms": random.uniform(3000, 8000),
            "request_count": random.randint(800, 1200)
        }
        httpx.post("http://localhost:8000/api/metrics/ingest", json=payload)
        time.sleep(2)
        print(f"Metric pushed ({i+1}/20)")
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--type", default="database_connection_leak")
    args = parser.parse_args()
    if args.type == "database_connection_leak":
        simulate_database_connection_leak()
```

---

11. API Reference

Method	Endpoint	Description
GET	/health	Server health check
GET	/api/incidents	List all incidents
GET	/api/incidents/{id}	Get incident details
POST	/api/metrics/ingest	Push new metrics
GET	/api/metrics/{service}	Get service metrics
POST	/api/agents/run-rca	Trigger RCA on incident
POST	/api/agents/run-recovery	Generate recovery plan

Method	Endpoint	Description
GET	/api/predictions/{service}	Get failure predictions
POST	/api/recovery/{id}/approve	Approve recovery action
POST	/api/recovery/{id}/execute	Execute recovery action
GET	/ws/metrics	WebSocket for live metrics

---

12. Environment Variables

Create a `.env` file in the backend folder:

```
# Database

DATABASE_URL=postgresql://postgres:password@localhost:5432/aegisops

# Redis

REDIS_URL=redis://localhost:6379

# AI APIs

OPENAI_API_KEY=your_openai_key_here

GOOGLE_API_KEY=your_gemini_key_here

# Monitoring

PROMETHEUS_URL=http://localhost:9090

# App Settings

AUTONOMOUS_MODE=false

ANOMALY_THRESHOLD=0.05

PREDICTION_WINDOW_MINUTES=30
```

---

13. MVP Scope for Hackathon

Focus only on these for a strong working demo:

Must Have (Core Demo)

- [ ] FastAPI backend with incidents and metrics API
- [ ] PostgreSQL with incidents + metrics tables
- [ ] Prometheus metrics ingestion
- [ ] Isolation Forest anomaly detection (real ML model)
- [ ] LLM-powered RCA agent (GPT-4 or Gemini)
- [ ] Recovery plan generation
- [ ] React dashboard with real-time updates
- [ ] Failure simulation script
- [ ] Docker Compose for one-click startup

Good to Have (Bonus Points)

- [ ] LSTM anomaly detection
- [ ] RAG with FAISS for past incidents
- [ ] Failure prediction dashboard
- [ ] Auto-execution of recovery in autonomous mode
- [ ] Incident report PDF generation

Skip for Now (Future Scope)

- [ ] Kubernetes deployment
- [ ] Security agent
- [ ] Multi-cloud support
- [ ] Digital twin simulation

Quick Start Commands

```
# Clone and setup

git clone https://github.com/yourteam/aegisops

cd aegisops
```

```
# Start all services
docker-compose up -d

# Install backend dependencies
cd backend
pip install -r requirements.txt
uvicorn main:app --reload

# Install frontend dependencies
cd frontend
npm install
npm run dev

# Run demo simulation
python scripts/simulate_failure.py --type database_connection_leak
```

AegisOps — Built for HackNest | Transforming DevOps from Reactive to Autonomous